PRÉSENTATION INTERNE · 20 MINUTES

Script de démo Graphify

Une carte du code, pas un dump de fichiers.
Démo live sur le projet perso gluco-scan
pour un pilote sur le gros projet d'équipe.

331	1 612	4 262	0
fichiers indexés	nœuds	arêtes	token LLM

À PRÉPARER AVANT LA RÉUNION

1. Navigateur — graphify-out/graph.html déjà ouvert
2. Terminal — cd gluco-scan, commandes prêtes à coller
3. Cursor — un chat Agent vide pour les prompts

github.com/Graphify-Labs/graphify

Comment utiliser ce script

Ce document est le fil conducteur de la démo live, après les slides. Objectif : montrer en 8 à 10 minutes que Graphify donne une carte du repo (query / path / explain) au lieu de relire des centaines de fichiers.

Phrase d'ouverture : « 331 fichiers, 1 612 nœuds, extraction code à 0 token. On va poser les mêmes questions qu'en revue d'architecture. »

Timing de la réunion (20 min)

Min	Quoi
0 – 2	Slides 1–3 · le problème (tokens, grep, mémoire volatile)
2 – 7	Slides 4–8 · l'outil, le graphe, l'intégration Cursor
7 – 10	Slides 9–12 · gros projet, limites, décision de pilote
10 – 18	Démo · actes 1 à 3 (visuel, CLI, prompts Cursor)
18 – 20	Questions · go / no-go du pilote 2 semaines

Trois fenêtres à ouvrir

Fenêtre	Rôle
Navigateur	graphify-out/graph.html — le wow visuel
Terminal	Racine de gluco-scan — commandes CLI collées telles quelles
Cursor (Agent)	Questions naturelles — la rule graphify.mdc force le graphe

Rappel chiffres gluco-scan

- Corpus : 331 fichiers · ~254 000 mots
- Graphe : 1 612 nœuds · 4 262 arêtes · 126 communautés
- Extraction code : 0 input · 0 output (AST local, 100 % EXTRACTED)
- God nodes à citer : formatDecimal(), ProductTag, Product, ProductFormSheet(), useTutorialStore

Acte 1 — Le wow visuel (1 min)

Montrer que le graphe existe déjà. Ne pas reconstruire en live.

```
cd /Users/mcamus/GitHub/gluco-scan
open graphify-out/graph.html
```

À pointer à l'écran

- Communautés colorées (clusters Leiden)
- Hubs : openFoodFacts.service.ts, ProductFormSheet.tsx, meal.store.ts
- Clic sur un nœud → voisins immédiats

> **Phrase à dire**
> *« Ça, c'est la carte. L'agent n'a plus à la reconstruire à chaque chat. »*

Option : ouvrir graphify-out/GRAPH_REPORT.md et scroller jusqu'à God Nodes. Ne pas s'attarder sur les Suggested Questions (parfois bruitées : react / dependencies).

Acte 2 — CLI live (4 min)

Enchaîner les 5 commandes sans improvisation. Si un path dirigé échoue, rappeler que --undirected ignore le sens des arêtes.

2.1 Explique-moi ce fichier hub

```
graphify explain "openFoodFacts.service.ts"
```

> **RÉSULTAT ATTENDU**
> degree 42
> imports depuis ProductFormSheet, ProductsTabLayout,
> ProductEanScanSheet, useMealProductScan, useOffProductSearch
> fonctions : fetchProductByEAN / fetchOffPartialByEAN

2.2 Chemin scan → formulaire (le plus spectaculaire)

```
graphify path "fetchProductByEAN" "ProductFormSheet.tsx" --undirected
```

> **RÉSULTAT ATTENDU**
> fetchProductByEAN()
> ← ProductsTabLayout.tsx
> → ProductFormSheet.tsx
> 2 hops, tout EXTRACTED. Pas de supposition.

2.3 Les glucides, du produit jusqu'aux stats

```
graphify path "computeMealItemCarbsWithCooking" "statisticsCompute.service.ts" --undirected
```

> **RÉSULTAT ATTENDU**
>
> ```
> computeMealItemCarbsWithCooking()
>   → hasCookingConversion()
>   ← cookedRaw.selectors.ts
>   ← statisticsCompute.service.ts
> ```

Phrase : cuisine / conversion → sélecteurs → statistiques.

2.4 Contexte borné = tokens bornés

```
graphify query "scan EAN Open Food Facts product form" --budget 400
```

> **RÉSULTAT ATTENDU**
>
> ```
> TRUNCATED: showing 11 of 541 nodes (~400-token budget)
> Le budget cappe le contexte. Monter à --budget 1500 si besoin.
> ```

Variante plus large :

```
graphify query "How does meal carb calculation work from product to statistics?" --budget 800
```

Démarre sur Meal, Product, carbs.ts, statisticsCompute.service.ts — pas sur 331 fichiers.

2.5 Bonus glucides

```
graphify explain "computeCarbs"
graphify path "computeCarbs" "Product" --undirected
```

> **RÉSULTAT ATTENDU**
>
> ```
> computeCarbs()  ←  carbs.ts  →  Product
> ```

Acte 3 — Prompts Cursor (4 min)

Ne pas coller les commandes CLI dans le chat. Coller des questions naturelles. La rule .cursor/rules/graphify.mdc (alwaysApply) force l'agent à passer par query / path / explain avant Grep ou Read.

Critère de succès : le premier tool call est graphify, pas grep.

Prompt 1 — Orientation (30 s)

```
Comment fonctionne le scan EAN et Open Food Facts dans ce projet ?
Ne lis pas tout le repo : commence par le graphe.
```

Prompt 2 — Chemin (le plus spectaculaire)

```
Trace le chemin entre fetchProductByEAN et ProductFormSheet.
Quels fichiers sont réellement sur le chemin ? Cite les source_location.
```

Prompt 3 — Impact d'un changement (cas équipe)

```
Si je change computeCarbs dans src/utils/carbs.ts, qu'est-ce qui casse ?
Donne les appelants et le chemin jusqu'aux statistiques repas.
```

Prompt 4 — Refacto / cohésion

```
Le rapport Graphify dit que la communauté statisticsCompute.service.ts
a une cohésion très faible (~0.06).
Est-ce que ce service devrait être splitté ? Propose un découpage
en t'appuyant uniquement sur le graphe, puis vérifie 2-3 fichiers.
```

Prompt 5 — Onboarding d'un collègue

```
Je débarque sur gluco-scan. En 10 bullets :
- les god nodes
- le flux création de repas
- où vit SQLite / les repositories
- où vit Open Food Facts
Pas de lecture exhaustive.
```

Prompt 6 — Contraste pédagogique (optionnel)

Deux chats séparés. Compter à voix haute : fichiers ouverts, temps, précision.

A — sans graphe (mauvais réflexe)

```
Ignore graphify. Trouve comment les glucides d'un repas sont calculés
en greppant le repo.
```

B — avec graphe

```
Même question, via graphify query / path / explain.
```

Acte 4 — Maintenance (30 s)

Rassurer l'équipe : le graphe ne pourrit pas si on l'update.

```
# Après un changement de code (0 token, AST only)
graphify update .

# Rebuild auto à chaque commit
graphify hook install

# Install Cursor sur un autre poste
uv tool install graphifyy
graphify cursor install
```

Sur gluco-scan, coût d'extraction = 0. Le LLM n'entre que si on indexe docs / PDF / images.

Fiche commandes — à garder sous la main

Install

```
uv tool install graphifyy
graphify cursor install          # rule Cursor alwaysApply
graphify claude install          # si l'équipe est sur Claude Code
```

Build

```
graphify .                       # premier graphe
graphify update .                # incrémental
open graphify-out/graph.html
```

Démo (copier-coller)

```
graphify explain "openFoodFacts.service.ts"
graphify explain "ProductFormSheet"
graphify explain "computeCarbs"
graphify path "fetchProductByEAN" "ProductFormSheet.tsx" --undirected
graphify path "computeMealItemCarbsWithCooking" "statisticsCompute.service.ts" --undirected
graphify path "computeCarbs" "Product" --undirected
graphify query "scan EAN Open Food Facts product form" --budget 400
graphify query "How does meal carb calculation work from product to statistics?" --budget 800
```

Si ça déraille en live

Symptôme	Réflexe
path : No directed path found	Relancer avec --undirected
query trop large / TRUNCATED	Baisser --budget ou resserrer les mots-clés
Agent qui greppe d'abord	Rappeler : « commence par le graphe » dans le prompt
Graphe visiblement stale	graphify update . (0 token)
Ambiguous node match	Utiliser le nom de fichier exact (ex. ProductFormSheet.tsx)

Clôture — la décision

Proposition : pilote 2 semaines sur un module du gros projet.

- Graphe + graph.html du module choisi
- Rule Cursor (.cursor/rules/graphify.mdc) commitée
- 5 questions d'archi avant / après (fichiers lus, tokens)
- Go / no-go en fin de pilote

Ne pas promettre

Le ×71,5 est un benchmark publié (corpus Karpathy), pas une garantie sur notre repo. On mesure pendant le pilote.

Repo de démo : gluco-scan · Outil : github.com/Graphify-Labs/graphify · Extraction code : AST local, zéro clé API.